Prerequisites

These notes collect the mathematical background assumed throughout the course. Nothing here is derived during lectures; if any section feels genuinely unfamiliar rather than rusty, treat it as a reading assignment before the corresponding lecture.

Calculus

The Derivative

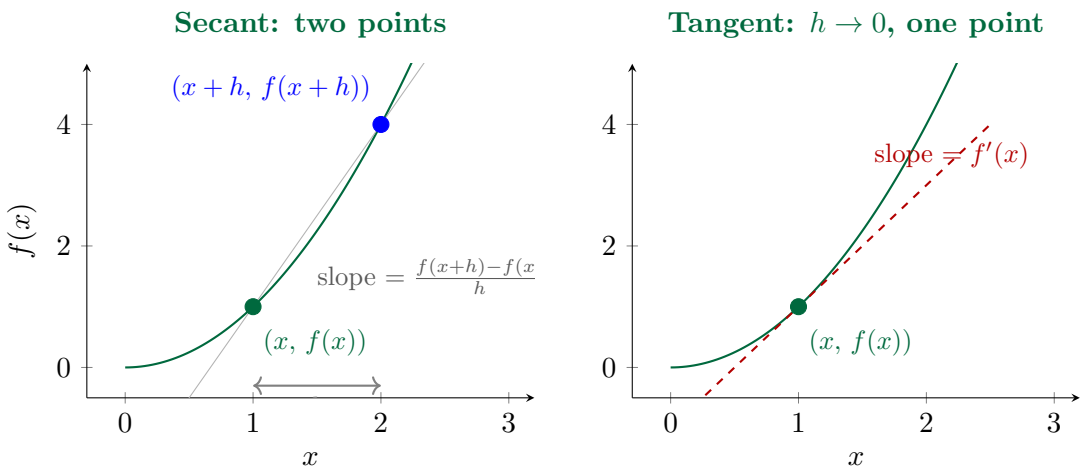
A function $f : \mathbb{R} \to \mathbb{R}$ assigns to each input x an output $f(x)$. The derivative $f'(x)$ is the instantaneous rate of change of f at x. To make this precise, fix a point x and consider a nearby point $x + h$: the ratio

$$\frac{f(x+h) - f(x)}{h}$$

is the average rate of change over the interval $[x, x+h]$, geometrically the slope of the **secant line** through $(x, f(x))$ and $(x+h, f(x+h))$. The derivative is the limit of this slope as h shrinks to zero:

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

As $h \to 0$, the secant line rotates into the **tangent line** at x; the derivative is the slope of that tangent. On $f(x) = x^2$ at $x = 1$: the left panel shows secants for large h; the right panel shows the secant approaching the tangent as h shrinks.



Standard rules. For differentiable f, g and constant c:

- $(f+g)' = f' + g'$; $\quad (fg)' = f'g + fg'$; $\quad (f/g)' = (f'g - fg')/g^2$
- **Chain rule**: $(f \circ g)'(x) = f'(g(x)) \cdot g'(x)$
- $(x^n)' = nx^{n-1}$; $\quad (e^x)' = e^x$; $\quad (\ln x)' = 1/x$

The chain rule is the single most important differentiation rule in the course: backpropagation in neural networks is nothing more than the chain rule applied to a composition of many functions.

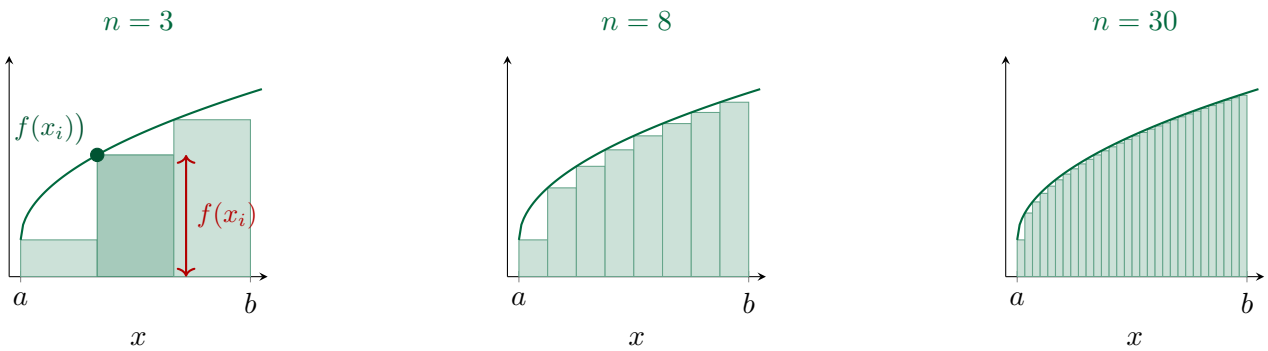
The Integral

Given a function f, the integral $\int_a^b f(x)\,dx$ is the signed area between f and the x-axis on $[a, b]$: the total quantity f accumulates over the interval. To compute it, partition $[a, b]$ into n subintervals of

width $\Delta x = (b-a)/n$, approximate the area in each slice by a rectangle of height $f(x_i^*)$ and width Δx, and sum:

$$S_n = \sum_{i=1}^{n} f(x_i^*)\,\Delta x$$

As n grows, Δx shrinks, the rectangles trace f more faithfully, and the error between the sum and the true area vanishes:



$$\underbrace{\sum_{i=1}^{n} f(x_i)\,\Delta x}_{\text{finite sum of rectangles}} \xrightarrow[\Delta x \to 0]{n \to \infty} \underbrace{\int_a^b f(x)\,dx}_{\text{exact area}}$$

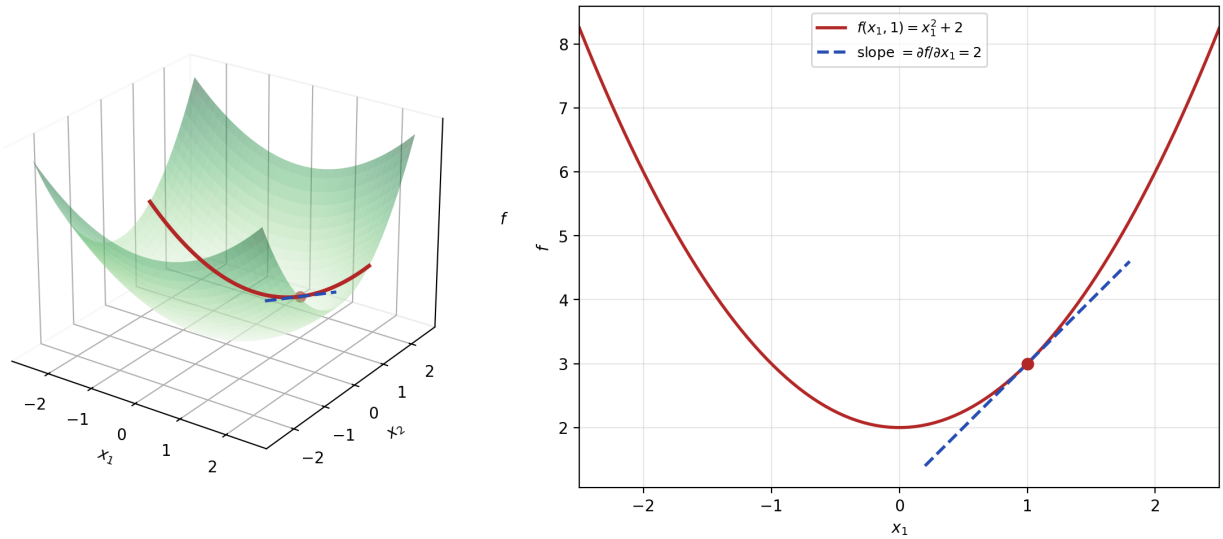
The **Fundamental Theorem of Calculus** connects differentiation and integration: if $F'(x) = f(x)$, then $\int_a^b f(x)\,dx = F(b) - F(a)$. They are inverse operations; this duality appears throughout the course, most directly in the relationship between the probability density function and the cumulative distribution function.

Partial Derivatives and the Gradient

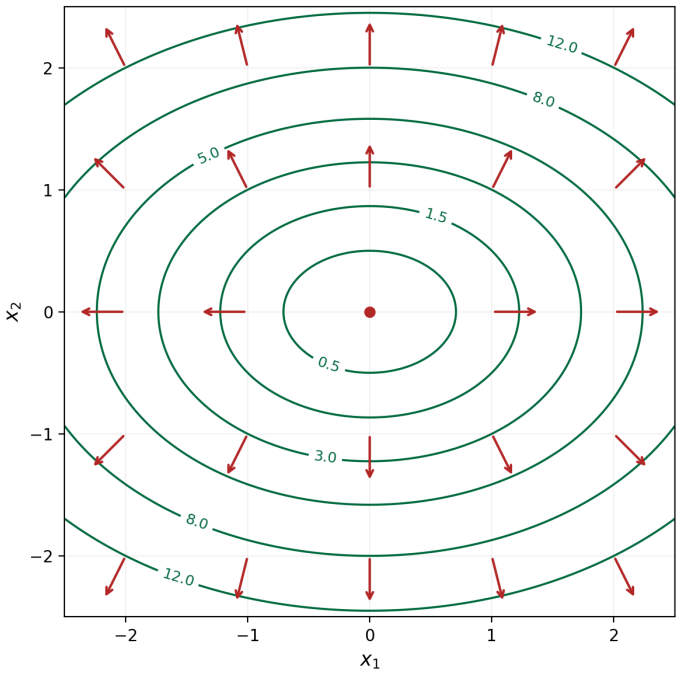
For $f : \mathbb{R}^d \to \mathbb{R}$, the partial derivative $\frac{\partial f}{\partial x_j}$ measures the rate of change of f along the j-th coordinate axis while all other coordinates are held fixed. The **gradient**

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_d} \end{bmatrix} \in \mathbb{R}^d$$

collects all d partial derivatives into a single vector. Geometrically, $\nabla f(\mathbf{x})$ points in the direction of steepest ascent of f at $\mathbf{x}$, and its magnitude $\|\nabla f\|$ is the rate of increase in that direction. The left panel below shows a 3D surface $f(x_1, x_2) = x_1^2 + 2x_2^2$ sliced at $x_2 = 1$: the partial derivative $\partial f/\partial x_1$ is the slope of the curve in that slice (the tangent line). The right panel extracts the 1D slice.



On a contour plot (curves of constant f), the gradient at each point is perpendicular to the level curve passing through that point. The left panel below shows gradient arrows on the 3D surface; the right shows the same from above, with gradient arrows cutting across contour lines at right angles.



Gradient descent exploits this directly: stepping in the direction $-\nabla f$ decreases f as rapidly as possible per unit step.

Critical Points, the Hessian, and Convexity

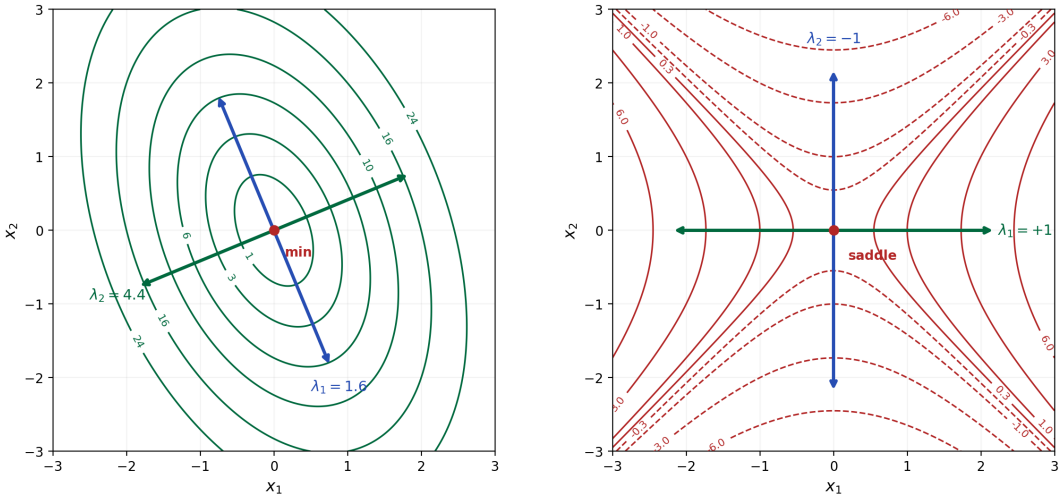
A necessary condition for $\mathbf{x}^*$ to be a local minimum of a differentiable $f : \mathbb{R}^d \to \mathbb{R}$ is $\nabla f(\mathbf{x}^*) = \mathbf{0}$ (the first-order condition). This is necessary but not sufficient: saddle points also satisfy it. To classify critical points, we examine the **Hessian**, the $d \times d$ matrix of second partial derivatives:

$$\mathbf{H}_f(\mathbf{x}) = \left[\frac{\partial^2 f}{\partial x_i \partial x_j} \right]_{i,j}$$

At a critical point $\nabla f(\mathbf{x}^*) = \mathbf{0}$: if $\mathbf{H}_f(\mathbf{x}^*)$ is positive definite (all eigenvalues positive), then $\mathbf{x}^*$ is a local minimum; if negative definite, a local maximum; otherwise, a saddle point.

Definition 1 (Convexity). *A function f is **convex** if $\mathbf{H}_f(\mathbf{x}) \succeq 0$ everywhere (equivalently, if $f(t\mathbf{a}+(1-t)\mathbf{b}) \leq t\,f(\mathbf{a})+(1-t)\,f(\mathbf{b})$ for all $\mathbf{a}, \mathbf{b}$ and $t \in [0,1]$). For convex functions, every local minimum is a global minimum, and the first-order condition $\nabla f(\mathbf{x}^*) = \mathbf{0}$ is both necessary and sufficient.*

Convexity is the reason linear regression has a unique, closed-form solution while neural network training does not: the squared-error loss is convex in the parameters of a linear model but non-convex in the parameters of a neural network.



Left: $\mathbf{H} \succ 0$ (both $\lambda > 0$): contours are ellipses, curvature is positive along every eigenvector direction $\Rightarrow$ bowl with unique minimum. Right: indefinite $\mathbf{H}$ ($\lambda_1 > 0, \lambda_2 < 0$): the positive-eigenvalue direction curves up, the negative-eigenvalue direction curves down $\Rightarrow$ saddle.

Linear Algebra

Matrices as Linear Transformations

A matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ defines a linear function $f : \mathbb{R}^n \to \mathbb{R}^m$ by $f(\mathbf{x}) = \mathbf{A}\mathbf{x}$. What this function does is determined by the columns of $\mathbf{A}$: the j-th column is the image of the j-th standard basis vector $\mathbf{e}_j$:

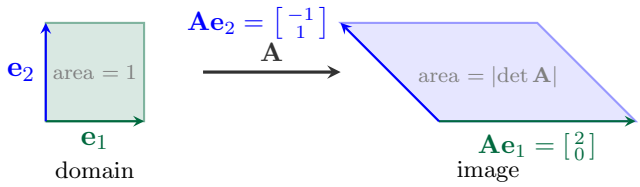
$$\mathbf{A}\mathbf{e}_j = \begin{bmatrix} a_{1j} \\ a_{2j} \\ \vdots \\ a_{mj} \end{bmatrix} = (\text{the } j\text{-th column of } \mathbf{A})$$

Now any vector $\mathbf{x} \in \mathbb{R}^n$ can be decomposed as $\mathbf{x} = x_1\mathbf{e}_1 + x_2\mathbf{e}_2 + \cdots + x_n\mathbf{e}_n$; by linearity,

$$\mathbf{A}\mathbf{x} = x_1\,(\mathbf{A}\mathbf{e}_1) + x_2\,(\mathbf{A}\mathbf{e}_2) + \cdots + x_n\,(\mathbf{A}\mathbf{e}_n)$$

The output is a linear combination of the columns of $\mathbf{A}$, weighted by the entries of $\mathbf{x}$. Once you know where each basis vector lands, you know where *every* vector lands, since every vector decomposes into basis vectors.

A concrete example helps. Let $\mathbf{A} = \begin{bmatrix} 2 & -1 \\ 0 & 1 \end{bmatrix}$ and $\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$. The first column $\begin{bmatrix} 2 \\ 0 \end{bmatrix}$ tells us $\mathbf{e}_1$ gets stretched horizontally by 2; the second column $\begin{bmatrix} -1 \\ 1 \end{bmatrix}$ tells us $\mathbf{e}_2$ gets sheared to the left. The output $\mathbf{A}\mathbf{x} = 1 \cdot \begin{bmatrix} 2 \\ 0 \end{bmatrix} + 2 \cdot \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$ is the linear combination of these transformed basis vectors, with $x_1 = 1$ and $x_2 = 2$ as weights.



Geometrically, the matrix maps the unit square (spanned by $\mathbf{e}_1, \mathbf{e}_2$) to the parallelogram spanned by the columns of $\mathbf{A}$. Every point inside the square moves to the corresponding point inside the parallelogram, and evenly-spaced points in the domain remain evenly spaced in the image. A linear transformation can scale, rotate, reflect, shear, and project, but it cannot bend, twist, or warp.

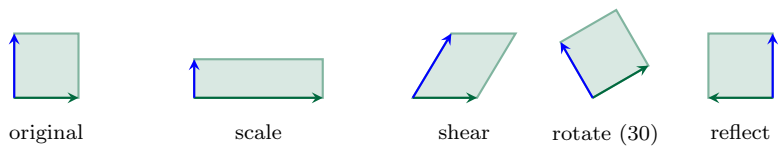
Types of Transformations

Different matrices produce different geometric effects, depending on their column structure.

Diagonal matrices scale each coordinate axis independently: $\begin{bmatrix}\alpha_1 & 0\\ 0 & \alpha_2\end{bmatrix}$ sends $\mathbf{e}_1 \mapsto \alpha_1\mathbf{e}_1$ and $\mathbf{e}_2 \mapsto \alpha_2\mathbf{e}_2$, stretching or compressing along each axis without mixing directions. The identity matrix is the special case $\alpha_1 = \alpha_2 = 1$; a negative α_j reflects across the corresponding axis.

Shear matrices have off-diagonal entries that make one coordinate's image depend on another. The matrix $\begin{bmatrix}1 & \beta\\ 0 & 1\end{bmatrix}$ leaves $\mathbf{e}_1$ fixed but pushes $\mathbf{e}_2$ sideways by β, tilting the unit square into a parallelogram. This is how skewing arises.

Rotation matrices $\mathbf{R}_\theta = \begin{bmatrix}\cos\theta & -\sin\theta\\ \sin\theta & \cos\theta\end{bmatrix}$ rotate every vector by angle θ counterclockwise; they preserve all lengths and angles (the columns are orthonormal). Any rotation can be decomposed into three shears, a fact exploited in fast image rotation algorithms.

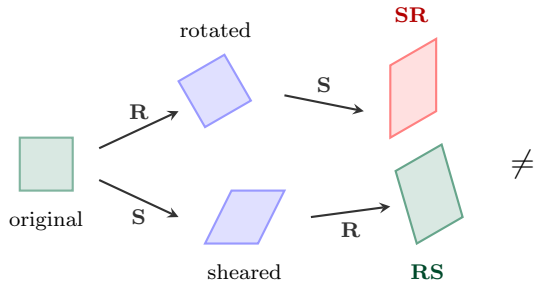


Any $m \times n$ matrix can be decomposed (via the singular value decomposition, or SVD) as a rotation, followed by a diagonal scaling, followed by another rotation. Every linear transformation reduces to rotations and axis-aligned stretches.

Composition and Non-Commutativity

Matrix multiplication corresponds to function composition: if $\mathbf{B}$ maps $\mathbb{R}^p \to \mathbb{R}^n$ and $\mathbf{A}$ maps $\mathbb{R}^n \to \mathbb{R}^m$, then $\mathbf{AB}$ maps $\mathbb{R}^p \to \mathbb{R}^m$ by first applying $\mathbf{B}$, then $\mathbf{A}$. The columns of $\mathbf{AB}$ are the images of the basis vectors under the composed map: column j of $\mathbf{AB}$ is $\mathbf{A}(\mathbf{B}\mathbf{e}_j)$, the result of first applying $\mathbf{B}$ to $\mathbf{e}_j$ and then applying $\mathbf{A}$.

An immediate consequence: matrix multiplication is generally not commutative. The diagram below makes this concrete. Start with the same unit square and apply a shear $\mathbf{S}$ followed by a rotation $\mathbf{R}$, then compare with $\mathbf{R}$ first and $\mathbf{S}$ second:



The two results are different because each transformation acts relative to the axes that exist *after* the previous one. Commutativity $\mathbf{AB} = \mathbf{BA}$ holds only in special cases, such as when both matrices are diagonal (scaling along the same axes commutes).

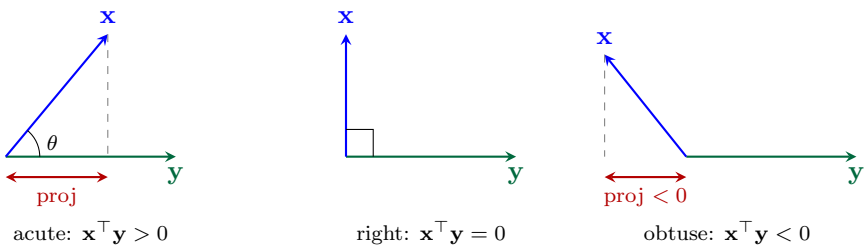
The Dot Product

The **dot product** (inner product) of $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ admits two equivalent definitions:

$$\mathbf{x}^\top \mathbf{y} = \sum_{i=1}^{d} x_i y_i = \|\mathbf{x}\| \, \|\mathbf{y}\| \cos\theta$$

where θ is the angle between the two vectors. The algebraic form is a sum of products; the geometric form is a product of lengths times the cosine of the angle between them.

Project $\mathbf{x}$ onto the line spanned by $\mathbf{y}$: the signed length of that projection is $\|\mathbf{x}\| \cos\theta$, and the dot product $\mathbf{x}^\top \mathbf{y}$ equals this projection length times $\|\mathbf{y}\|$. Three cases arise:

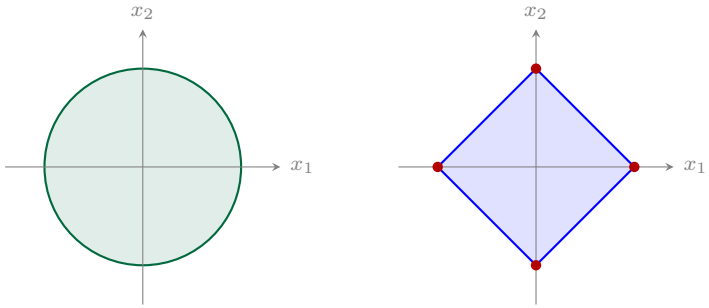


When $\theta < 90$, the vectors point in a similar direction and the dot product is positive. When $\theta = 90$, the projection vanishes and the dot product is zero. When $\theta > 90$, the projection falls behind the origin and the dot product is negative. For a PSD matrix $\mathbf{A}$, the condition $\mathbf{v}^\top \mathbf{A}\mathbf{v} \geq 0$ means the angle between $\mathbf{v}$ and $\mathbf{A}\mathbf{v}$ is never obtuse.

Norms

A **norm** $\|\mathbf{x}\|$ assigns a notion of "size" to each vector. Different norms measure size differently, and the difference matters for regularization.

The **Euclidean** (ℓ_2) **norm** $\|\mathbf{x}\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_d^2}$ measures straight-line length from the origin. The ℓ_1 **norm** $\|\mathbf{x}\|_1 = |x_1| + |x_2| + \cdots + |x_d|$ sums absolute values ("taxicab distance"). The difference is visible in their *unit balls*, the set of all vectors with $\|\mathbf{x}\| \leq 1$:



ℓ_2: circle ℓ_1: diamond (corners on axes)

The ℓ_1 ball has sharp corners that sit on the coordinate axes. When a regularization constraint (like $\|\boldsymbol{\theta}\|_1 \leq t$) forces the solution onto the boundary of this ball, those corners naturally push coefficients to be exactly zero, producing sparse solutions. The ℓ_2 ball is smooth; its boundary has no preferred directions, so coefficients shrink toward zero but rarely reach it. This geometric distinction is the reason Lasso (ℓ_1) produces sparse models while Ridge (ℓ_2) produces small-but-dense ones.

Rank, Determinant, and Invertibility

These three concepts quantify how much of the output space a transformation actually "uses."

Rank and the null space. The **rank** of $\mathbf{A} \in \mathbb{R}^{m\times n}$ is the dimension of its image: the set of all possible outputs $\{\mathbf{Ax} : \mathbf{x} \in \mathbb{R}^n\}$. Equivalently, it counts how many of the columns of $\mathbf{A}$ are linearly independent. When $\text{rank}(\mathbf{A}) = r < n$, the transformation collapses an n-dimensional input space onto an r-dimensional subspace; some directions in the input are completely lost.

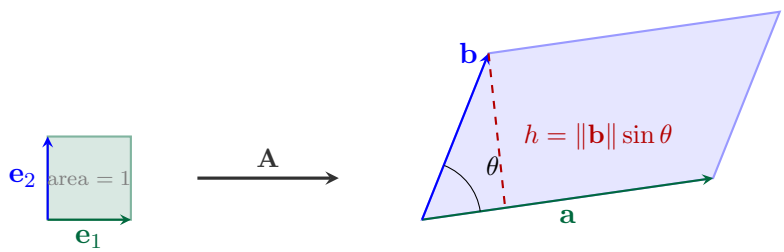
The directions that are lost form the **null space** $\ker(\mathbf{A}) = \{\mathbf{x} : \mathbf{Ax} = \mathbf{0}\}$: all inputs that get mapped to zero. Its dimension is $n - r$. Consider a 2×2 matrix with rank 1: both columns point in the same direction, so the entire 2D plane gets flattened onto a single line. Every vector perpendicular to that line is in the null space.

A rank-deficient transformation sends multiple distinct inputs to the same output, so it cannot be inverted; the information along the null space is lost.

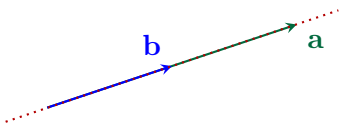
Determinant. The **determinant** of a transformation is the factor by which it scales areas (in 2D) or volumes (in higher dimensions). Consider the 2×2 matrix $\mathbf{A} = \begin{bmatrix} a_1 & b_1 \\ a_2 & b_2 \end{bmatrix}$ with columns $\mathbf{a}$ and $\mathbf{b}$. The unit square (spanned by $\mathbf{e}_1, \mathbf{e}_2$, area $= 1$) maps to the parallelogram spanned by $\mathbf{a}$ and $\mathbf{b}$. The area of that parallelogram is base times height: take $\mathbf{a}$ as the base (length $\|\mathbf{a}\|$) and drop a perpendicular from $\mathbf{b}$ to $\mathbf{a}$. The height is $\|\mathbf{b}\| \sin\theta$, where θ is the angle between the columns. The signed area is

$$\text{area} = a_1 b_2 - a_2 b_1 = \det(\mathbf{A})$$

This quantity is the determinant: the factor by which the transformation stretches or compresses area.



When $\det(\mathbf{A}) = 0$, the columns $\mathbf{a}$ and $\mathbf{b}$ are parallel; the parallelogram collapses to a line and the area is zero:



det = 0: $\mathbf{b} \parallel \mathbf{a}$, area = 0

When $\det(\mathbf{A}) > 0$, the columns $\mathbf{a}, \mathbf{b}$ span a parallelogram with positive area and the orientation (handedness) of the original square is preserved. When $\det(\mathbf{A}) < 0$, the area is the same but the orientation is flipped (a reflection). When $\det(\mathbf{A}) = 0$, the columns are parallel, the parallelogram collapses to a line, and the transformation is not invertible; you cannot recover the input from the output because an entire dimension of information has been lost.

In d dimensions, $\det(\mathbf{A})$ is the signed volume of the d-dimensional parallelepiped spanned by the d columns of $\mathbf{A}$.

Properties. The determinant is multiplicative: $\det(\mathbf{AB}) = \det(\mathbf{A})\det(\mathbf{B})$, which is geometrically natural: composing two transformations multiplies their volume scaling factors. It also equals the product of eigenvalues: $\det(\mathbf{A}) = \prod_j \lambda_j$.

Invertibility. A square matrix $\mathbf{A}$ is **invertible** if and only if $\det(\mathbf{A}) \neq 0$, equivalently $\operatorname{rank}(\mathbf{A}) = d$, equivalently the null space is trivial ($\ker(\mathbf{A}) = \{\mathbf{0}\}$). Geometrically: if no dimension is collapsed, the transformation can be reversed. The inverse $\mathbf{A}^{-1}$ satisfies $\mathbf{AA}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}_d$ and is unique. Key identity: $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$ (undo the second transformation first, then the first).

In linear regression, $\mathbf{X}^\top\mathbf{X}$ must be invertible for a unique OLS solution, which requires $\operatorname{rank}(\mathbf{X}) = d$ and hence $n \geq d$. When $\operatorname{rank}(\mathbf{X}) < d$, there exist directions $\mathbf{v} \neq \mathbf{0}$ with $\mathbf{Xv} = \mathbf{0}$, so infinitely many parameter vectors produce the same predictions.

Orthogonal Matrices

A square matrix $\mathbf{Q} \in \mathbb{R}^{d\times d}$ is **orthogonal** if its columns are mutually orthogonal unit vectors, which is equivalent to $\mathbf{Q}^\top\mathbf{Q} = \mathbf{QQ}^\top = \mathbf{I}$ (so $\mathbf{Q}^{-1} = \mathbf{Q}^\top$, the cheapest possible inverse). To see why columns must be orthonormal, write it out: the (i, j) entry of $\mathbf{Q}^\top\mathbf{Q}$ is $\mathbf{q}_i^\top\mathbf{q}_j$, which must be 1 when $i = j$ (unit length) and 0 otherwise (orthogonality).

Three geometric properties follow from a single algebraic trick: $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$.

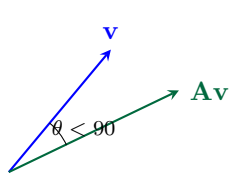
1. **Lengths are preserved**: $\|\mathbf{Qv}\|^2 = (\mathbf{Qv})^\top(\mathbf{Qv}) = \mathbf{v}^\top\mathbf{Q}^\top\mathbf{Qv} = \mathbf{v}^\top\mathbf{v} = \|\mathbf{v}\|^2$.
2. **Angles are preserved**: $(\mathbf{Qu})^\top(\mathbf{Qv}) = \mathbf{u}^\top\mathbf{v}$, so the cosine of the angle between any two vectors is unchanged.
3. **Volumes are preserved**: $\det(\mathbf{Q}^\top\mathbf{Q}) = \det(\mathbf{I}) = 1$ gives $\det(\mathbf{Q})^2 = 1$, so $\det(\mathbf{Q}) = \pm 1$. When $+1$, $\mathbf{Q}$ is a pure rotation; when -1, it includes a reflection.

An orthogonal matrix can rotate and reflect but cannot stretch, compress, or shear. Orthogonal matrices appear in the eigendecomposition $\mathbf{A} = \mathbf{Q\Lambda Q}^\top$ of a symmetric matrix: rotate axes to align with the eigenvectors ($\mathbf{Q}^\top$), scale each axis by the corresponding eigenvalue ($\mathbf{\Lambda}$), rotate back ($\mathbf{Q}$). The $\mathbf{Q}$ parts preserve geometry; all the stretching happens in $\mathbf{\Lambda}$.

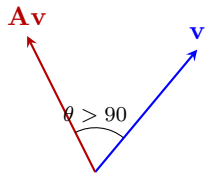
Positive Definiteness

A symmetric matrix $\mathbf{A} \in \mathbb{R}^{d\times d}$ is **positive definite** ($\mathbf{A} \succ 0$) if $\mathbf{v}^\top\mathbf{Av} > 0$ for all $\mathbf{v} \neq \mathbf{0}$, and **positive semi-definite** ($\mathbf{A} \succeq 0$) if $\mathbf{v}^\top\mathbf{Av} \geq 0$.

Start with the scalar case. If $a > 0$, then $b(ab) = ab^2 > 0$ for every $b \neq 0$; a positive number stretches b in the direction b already points, never flipping it past the origin. Now consider $\mathbf{v}^\top(\mathbf{Av})$: this is the dot product of $\mathbf{v}$ with its image $\mathbf{Av}$, so by our earlier discussion of the dot product, the condition $\mathbf{v}^\top\mathbf{Av} > 0$ says exactly that the angle between $\mathbf{v}$ and $\mathbf{Av}$ is always acute.



PD: acute angle $\Rightarrow \mathbf{v}^\top \mathbf{A}\mathbf{v} > 0$



Indefinite: obtuse $\Rightarrow \mathbf{v}^\top \mathbf{A}\mathbf{v} < 0$

Geometrically: a positive definite matrix never sends a vector past 90 from where it started. Positive semi-definiteness allows the boundary case (angle exactly 90, i.e., the vector is sent into a lower-dimensional subspace) but never past it.

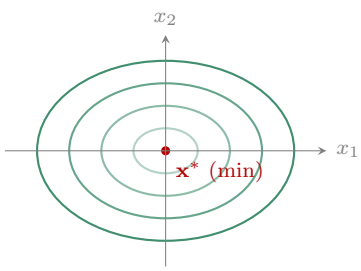
Eigenvalues. Since $\mathbf{A}\mathbf{v}_j = \lambda_j \mathbf{v}_j$ for each eigenvector, the image of $\mathbf{v}_j$ points in the same direction when $\lambda_j > 0$ and in the opposite direction when $\lambda_j < 0$. A negative eigenvalue reverses an eigenvector, sending it past 90, so positive definiteness requires all eigenvalues strictly positive and positive semi-definiteness requires all eigenvalues nonnegative. Since all eigenvalues are nonnegative, $\det(\mathbf{A}) = \prod_j \lambda_j \geq 0$ for PSD matrices.

Decomposition as a square root. Just as every nonneg real a has a square root $a = (\sqrt{a})^2$, a symmetric PSD matrix admits a factorization $\mathbf{A} = \mathbf{B}^\top \mathbf{B}$, which immediately proves $\mathbf{A} \succeq 0$ since $\mathbf{v}^\top \mathbf{B}^\top \mathbf{B}\mathbf{v} = \|\mathbf{B}\mathbf{v}\|^2 \geq 0$. Conversely, the eigendecomposition $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ with $\mathbf{\Lambda} \geq 0$ gives $\mathbf{B} = \mathbf{\Lambda}^{1/2}\mathbf{Q}^\top$. The Cholesky factorization $\mathbf{A} = \mathbf{L}\mathbf{L}^\top$ (lower-triangular $\mathbf{L}$) is the computationally preferred variant of this square root.

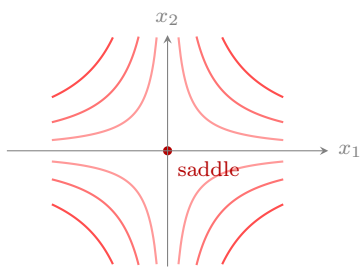
Why PSD guarantees a unique minimum. Consider the quadratic form $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x} - \mathbf{b}^\top \mathbf{x}$. Setting $\nabla f = \mathbf{A}\mathbf{x} - \mathbf{b} = \mathbf{0}$ gives the critical point $\mathbf{x}^* = \mathbf{A}^{-1}\mathbf{b}$. To see why this is a minimum, perturb it by any $\mathbf{e} \neq \mathbf{0}$:

$$f(\mathbf{x}^* + \mathbf{e}) = f(\mathbf{x}^*) + \tfrac{1}{2}\,\mathbf{e}^\top \mathbf{A}\,\mathbf{e}$$

(the cross terms cancel because $\mathbf{A}\mathbf{x}^* = \mathbf{b}$). When $\mathbf{A} \succ 0$, the perturbation term $\mathbf{e}^\top \mathbf{A}\,\mathbf{e} > 0$, so *every* direction away from $\mathbf{x}^*$ increases the objective: a bowl with a single bottom. When $\mathbf{A}$ is indefinite, some eigenvalues are negative, so $\mathbf{e}^\top \mathbf{A}\,\mathbf{e} < 0$ along those eigenvectors: the critical point is a saddle, not a minimum.



$\mathbf{A} \succ 0$: bowl (unique minimum)

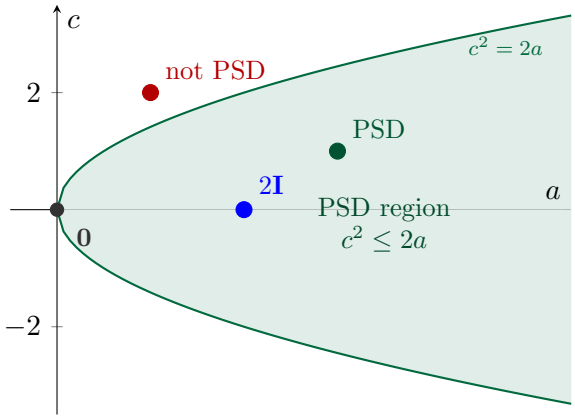


Indefinite: saddle (no minimum)

The Hessian of the OLS objective is $\frac{2}{n}\mathbf{X}^\top \mathbf{X}$, which is PSD since $\mathbf{v}^\top \mathbf{X}^\top \mathbf{X}\mathbf{v} = \|\mathbf{X}\mathbf{v}\|^2 \geq 0$, and strictly PD when $\mathbf{X}$ has full column rank. The OLS loss surface is therefore a bowl with a unique global minimum.

The PSD cone. Start with the scalar analogy. The nonnegative reals $[0, \infty)$ form a *ray* starting at the origin: scale any nonneg number by a nonneg constant, you get a nonneg number; add two nonneg numbers, you get a nonneg number. The set is closed under nonneg combinations. Now ask: what does the analogous set look like for matrices?

A 2×2 symmetric matrix $\begin{bmatrix} a & c \\ c & b \end{bmatrix}$ is described by three numbers (a, b, c), so we can plot the set of all such matrices as points in $\mathbb{R}^3$. The PSD conditions are $a \geq 0$, $b \geq 0$, and $\det = ab - c^2 \geq 0$. The boundary surface $ab = c^2$ is literally a cone: at "height" $a = b = k$, the allowed off-diagonal values are $|c| \leq k$, so the cross-section is an interval $[-k, k]$ that widens linearly as k grows.



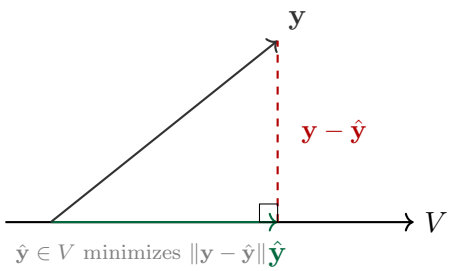
Cross-section at $b = 2$: the shaded region is the set of PSD matrices $\left[\begin{smallmatrix} a & c \\ c & 2 \end{smallmatrix}\right]$ with $c^2 \leq 2a$.

The identity matrix $\mathbf{I}$ sits at $(1, 1, 0)$ inside the cone; the zero matrix is the tip at the origin. Every PSD matrix is a point inside or on the surface. If you pick two points inside the cone (two PSD matrices $\mathbf{A}$ and $\mathbf{B}$) and draw the line segment between them, every point on that segment ($\alpha\mathbf{A} + (1-\alpha)\mathbf{B}$ for $\alpha \in [0, 1]$) is also inside the cone. That is convexity. If you also scale either point by any $\alpha \geq 0$ (stretching the ray from the origin through that point), you stay inside. That is what makes it a cone rather than just a convex set.

Proof: $\mathbf{v}^\top(\alpha\mathbf{A} + \beta\mathbf{B})\mathbf{v} = \alpha\,\mathbf{v}^\top\mathbf{A}\mathbf{v} + \beta\,\mathbf{v}^\top\mathbf{B}\mathbf{v} \geq 0$ whenever $\alpha, \beta \geq 0$ and $\mathbf{A}, \mathbf{B} \succeq 0$.

Orthogonal Projection

Given a subspace $V \subset \mathbb{R}^n$ and a vector $\mathbf{y}$, the **orthogonal projection** of $\mathbf{y}$ onto V is the unique point $\hat{\mathbf{y}} \in V$ closest to $\mathbf{y}$ in Euclidean distance. The residual $\mathbf{y} - \hat{\mathbf{y}}$ is perpendicular to every vector in V; this is both the definition and the optimality condition.



Why does the shortest path from a point to a plane always hit the plane at a right angle? Because any other path through V introduces a component along V that only adds to the distance. The perpendicularity condition $\hat{\mathbf{y}} \perp (\mathbf{y} - \hat{\mathbf{y}})$ can be verified via the Pythagorean theorem: for any other $\mathbf{z} \in V$, $\|\mathbf{y} - \mathbf{z}\|^2 = \|\mathbf{y} - \hat{\mathbf{y}}\|^2 + \|\hat{\mathbf{y}} - \mathbf{z}\|^2 \geq \|\mathbf{y} - \hat{\mathbf{y}}\|^2$.

OLS does exactly this. The fitted values $\hat{\mathbf{y}} = \mathbf{X}\hat{\boldsymbol{\theta}}$ are the orthogonal projection of the observed $\mathbf{y}$ onto the column space of $\mathbf{X}$; the normal equations $\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\theta}}) = \mathbf{0}$ state that the residual is perpendicular to every column of $\mathbf{X}$.

Probability

From Counting to Density

In a finite sample space Ω with equally likely outcomes, the probability of an event $A \subseteq \Omega$ is the ratio

$$P(A) = \frac{|A|}{|\Omega|}$$

which counts favorable outcomes and divides by the total. This works because each individual outcome carries a definite positive probability $1/|\Omega|$.

In the continuous case, this counting approach breaks down entirely. If X takes values in $\mathbb{R}$, then $P(X = x) = 0$ for every single x: uncountably many points must share a total probability of 1, and no individual point can carry positive mass. We cannot count points, so we replace counting with *density*. A **probability density function** (PDF) $f : \mathbb{R} \to [0, \infty)$ assigns to each point x a *density*, a probability per unit length, such that the probability of X falling in an interval $[a, b]$ is recovered by integrating:

$$P(a \leq X \leq b) = \int_a^b f(x)\, dx$$

The transition from discrete to continuous is precisely the transition from sums to integrals, from mass functions to density functions. In the discrete case we sum $p(x)$ over the favorable values; in the continuous case we integrate $f(x)$ over the favorable region. The quantity $f(x)\, dx$ is the probability contained in an infinitesimal slice $[x, x + dx]$, exactly as in a Riemann sum: chop the region into thin slices of width dx, assign each slice a probability proportional to the local density $f(x)$, and sum.

Note that $f(x)$ itself is not a probability; it can exceed 1 (consider a Uniform$(0, 0.1)$ distribution, where $f(x) = 10$ on $(0, 0.1)$). What must equal 1 is the total integral: $\int_{-\infty}^{\infty} f(x)\, dx = 1$.

CDF, PDF, and Calculus

The **cumulative distribution function** (CDF) $F(x) = P(X \leq x) = \int_{-\infty}^{x} f(t)\, dt$ accumulates density from the left. It is nondecreasing, with $F(-\infty) = 0$ and $F(\infty) = 1$. Differentiation inverts integration: $f(x) = F'(x)$. The PDF is the derivative of the CDF; the CDF is the integral of the PDF.

Expectation

The **expectation** of X is its density-weighted average:

$$\mathbb{E}[X] = \begin{cases} \sum_x x\, p(x) & \text{(discrete: sum of values weighted by mass)} \\ \int_{-\infty}^{\infty} x\, f(x)\, dx & \text{(continuous: integral of values weighted by density)} \end{cases}$$

Linearity of expectation: $\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$, regardless of whether X and Y are independent. No assumptions about dependence are needed.

Variance, Covariance, and the Covariance Matrix

The **variance** $\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$ measures dispersion around the mean. It is always nonnegative, since it is an expectation of a squared quantity.

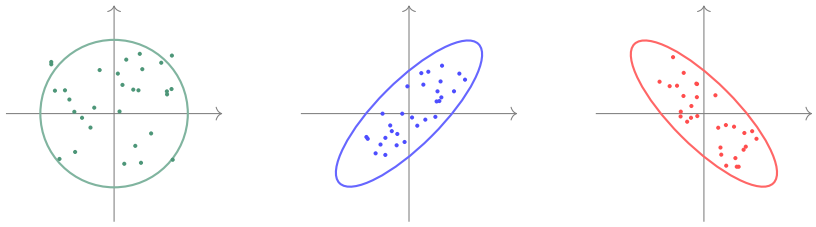
Properties:

- $\mathrm{Var}(aX + b) = a^2\mathrm{Var}(X)$ (shifting by b does not change spread)
- $\mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) + 2\mathrm{Cov}(X, Y)$

- If X, Y independent: $\mathrm{Var}(X+Y) = \mathrm{Var}(X) + \mathrm{Var}(Y)$

The **covariance** $\mathrm{Cov}(X,Y) = \mathbb{E}[(X-\mathbb{E}[X])(Y-\mathbb{E}[Y])] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]$ measures the linear association between X and Y: positive when they tend to move together, negative when they move oppositely, zero when there is no linear relationship. The **correlation** $\rho_{X,Y} = \mathrm{Cov}(X,Y)/(\sigma_X\,\sigma_Y) \in [-1,1]$ is the scale-free version.

When $\rho = 0$, the scatter cloud is circular (no preferred direction); as $|\rho| \to 1$, it elongates into an ellipse aligned with the diagonal:



$\rho = 0$ $\rho \approx +0.8$ $\rho \approx -0.8$

Correlation as cosine of angle. The correlation coefficient has a clean geometric meaning beyond these scatter plots. Given n observations $(x_1,y_1),\ldots,(x_n,y_n)$, center both variables: $\tilde{x}_i = x_i - \bar{x}$ and $\tilde{y}_i = y_i - \bar{y}$. Now treat $\tilde{\mathbf{x}} = (\tilde{x}_1,\ldots,\tilde{x}_n)$ and $\tilde{\mathbf{y}} = (\tilde{y}_1,\ldots,\tilde{y}_n)$ as vectors in $\mathbb{R}^n$. The sample correlation is

$$r = \frac{\sum_i \tilde{x}_i\,\tilde{y}_i}{\sqrt{\sum_i \tilde{x}_i^2}\,\sqrt{\sum_i \tilde{y}_i^2}} = \frac{\tilde{\mathbf{x}}\cdot\tilde{\mathbf{y}}}{\|\tilde{\mathbf{x}}\|\,\|\tilde{\mathbf{y}}\|} = \cos\theta$$

where θ is the angle between the two centered data vectors. Parallel vectors ($\theta = 0$) give $r = 1$; orthogonal vectors ($\theta = 90$) give $r = 0$; antiparallel vectors ($\theta = 180$) give $r = -1$. The coefficient of determination $R^2 = \cos^2\theta$ is the fraction of one variable's variance explained by the other: at 45, exactly half the variance is explained; below 30, at least 75%.

In OLS, the fitted values $\hat{\mathbf{y}}$ are the orthogonal projection of $\mathbf{y}$ onto the column space of $\mathbf{X}$, and the multiple R is the cosine of the angle between the centered response $\mathbf{y} - \bar{y}\mathbf{1}$ and its projection.

$\mathrm{Cov}(X,Y) = 0$ does not imply independence (e.g., $X \sim \mathrm{Uniform}(-1,1)$ and $Y = X^2$ are uncorrelated but perfectly dependent). Independence always implies zero covariance, but correlation measures only *linear* dependence.

The covariance matrix as high-dimensional variance. For a random vector $\mathbf{X} \in \mathbb{R}^d$, the **covariance matrix**

$$\mathbf{\Sigma} = \mathbb{E}[(\mathbf{X}-\mathbb{E}[\mathbf{X}])(\mathbf{X}-\mathbb{E}[\mathbf{X}])^\top] \in \mathbb{R}^{d\times d}$$

is the natural multivariate generalization of scalar variance: the diagonal entries $\Sigma_{ii} = \sigma_i^2$ are the variances of individual components, and the off-diagonal entries $\Sigma_{ij} = \rho_{ij}\sigma_i\sigma_j$ are the pairwise covariances. This structure yields a clean factorization:

$$\mathbf{\Sigma} = \mathrm{diag}(\boldsymbol{\sigma})\ \mathbf{C}\ \mathrm{diag}(\boldsymbol{\sigma})$$

where $\mathrm{diag}(\boldsymbol{\sigma})$ is the diagonal matrix of standard deviations and $\mathbf{C}$ is the correlation matrix (ones on the diagonal, ρ_{ij} off-diagonal). The covariance matrix encodes two things: the marginal dispersions σ_i and the linear dependencies ρ_{ij} between components.

The covariance matrix is always PSD, since for any fixed $\mathbf{v}$,

$$\mathbf{v}^\top\mathbf{\Sigma}\mathbf{v} = \mathbb{E}\Big[(\mathbf{v}^\top(\mathbf{X}-\boldsymbol{\mu}))^2\Big] \geq 0$$

which is the multivariate analog of $\sigma^2 \geq 0$. Variance properties lift to matrices: $\mathrm{Var}(\mathbf{AX}) = \mathbf{A\Sigma A}^\top$ (analogous to $\mathrm{Var}(aX) = a^2\sigma^2$), and adding a constant vector does not change dispersion ($\mathrm{Var}(\mathbf{a}+\mathbf{X}) = \mathbf{\Sigma}$, analogous to $\mathrm{Var}(a+X) = \sigma^2$).

Cholesky and whitening. The Cholesky factorization $\mathbf{\Sigma} = \mathbf{L}\mathbf{L}^\top$ is the multivariate square root, with $\mathbf{L}$ playing the role of σ. In the scalar case, the z-score $Z = (X - \mu)/\sigma$ standardizes to zero mean and unit variance; the multivariate analog is the **whitening transformation** $\mathbf{Z} = \mathbf{L}^{-1}(\mathbf{X} - \boldsymbol{\mu})$, which produces components with identity covariance $\mathrm{Var}(\mathbf{Z}) = \mathbf{I}$. Similarly, the precision matrix $\mathbf{\Sigma}^{-1}$ is the multivariate analog of $1/\sigma^2$, and the Mahalanobis distance $(\mathbf{x} - \boldsymbol{\mu})^\top \mathbf{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})$ is a multivariate squared z-score, which is exactly the exponent in the multivariate Gaussian density.

Independence and Conditional Expectation

Random variables X and Y are **independent** if knowing X tells you nothing about Y: $P(X \in A, Y \in B) = P(X \in A)\, P(Y \in B)$ for all sets A, B. A sequence $X_1, \ldots, X_n$ is **i.i.d.** (independent and identically distributed) if the X_i are mutually independent and share the same distribution; this is the standing assumption for training data throughout the course.

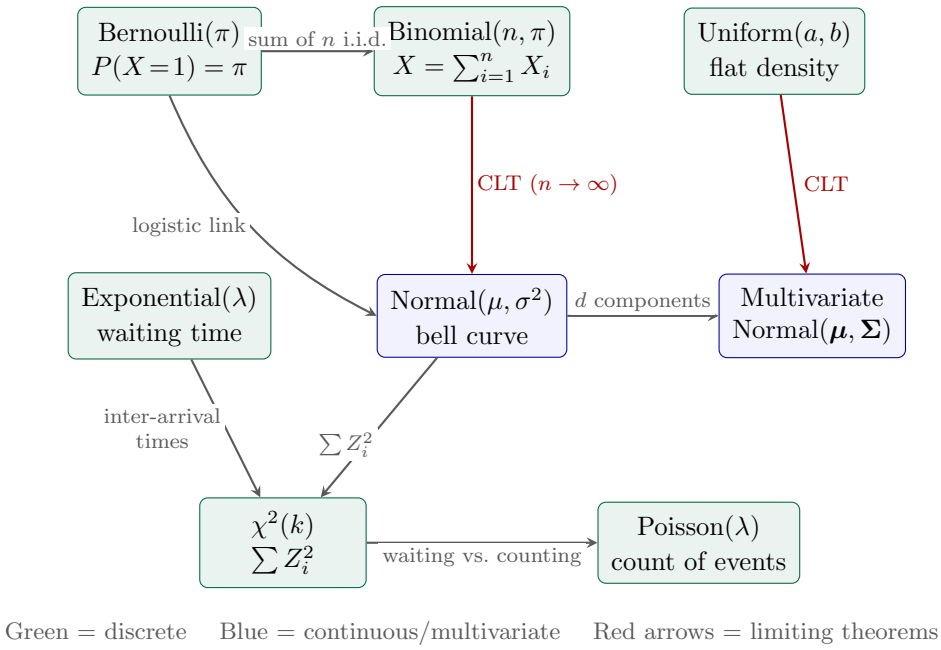
The **conditional expectation** $\mathbb{E}[Y \mid X = x]$ is the expected value of Y given that $X = x$; it is a function of x. Key formulas:

- **Law of Total Expectation**: $\mathbb{E}[Y] = \mathbb{E}_X[\mathbb{E}[Y \mid X]]$
- **Law of Total Variance**: $\mathrm{Var}(Y) = \mathbb{E}_X[\mathrm{Var}(Y \mid X)] + \mathrm{Var}_X(\mathbb{E}[Y \mid X])$

The conditional expectation $\mathbb{E}[Y \mid X]$ is the best predictor of Y from X under squared loss, a fact proved in the linear regression notes and the fundamental reason we use squared error.

Key Distributions

The distributions used in the course are connected by summation and limiting operations. The map below shows the relationships; the subsequent figures visualize the most important one.



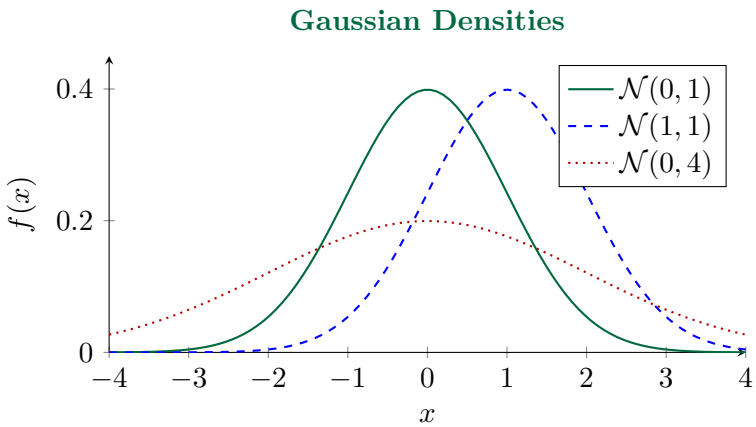
Bernoulli(π): models a binary outcome, $P(X = 1) = \pi$, $P(X = 0) = 1 - \pi$, with $\mathbb{E}[X] = \pi$ and $\mathrm{Var}(X) = \pi(1 - \pi)$. The building block for logistic regression.

Gaussian (Normal) $\mathcal{N}(\mu, \sigma^2)$: the bell curve. Its density is

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

with $\mathbb{E}[X] = \mu$, $\mathrm{Var}(X) = \sigma^2$. The exponent is the squared deviation from μ normalized by σ^2, so

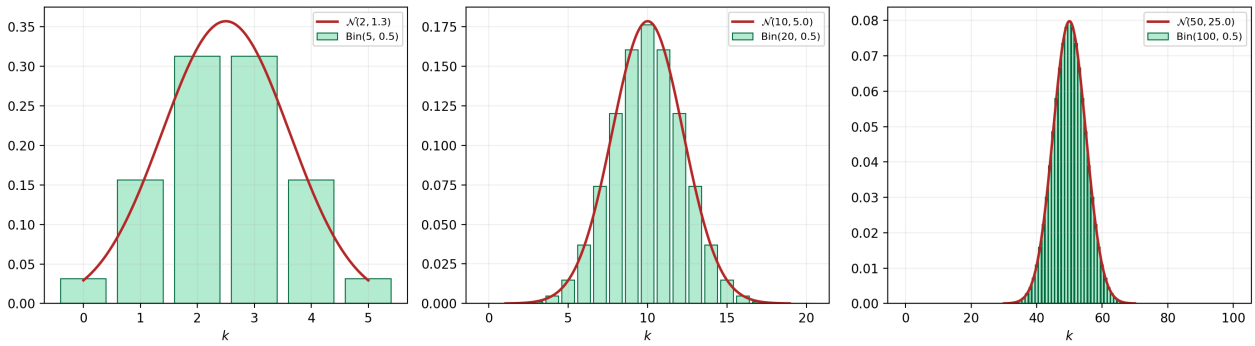
minimizing squared error and maximizing the Gaussian log-likelihood are the same problem. Linear combinations of independent Gaussians are Gaussian: $aX + bY \sim \mathcal{N}(a\mu_X + b\mu_Y,\, a^2\sigma_X^2 + b^2\sigma_Y^2)$.



De Moivre–Laplace theorem. The sum of n i.i.d. Bernoulli(π) random variables is Binomial(n, π). As n grows, the standardized Binomial converges to the standard Normal:

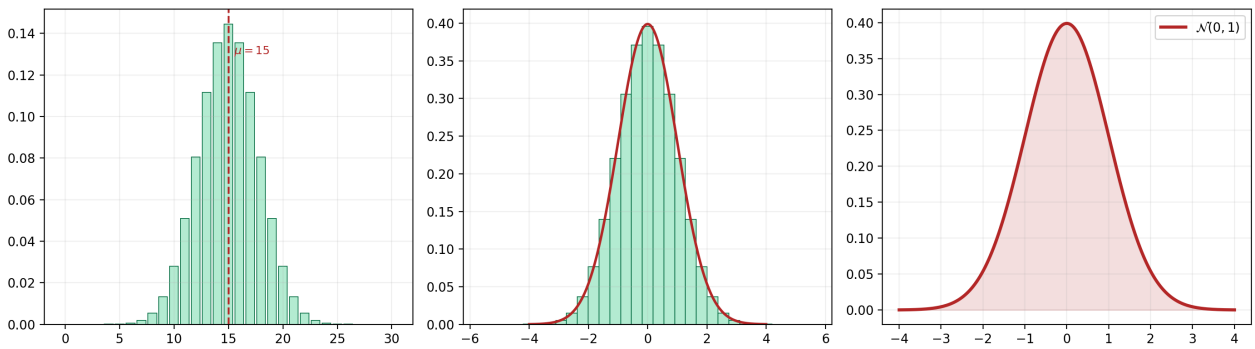
$$\frac{X - n\pi}{\sqrt{n\pi(1-\pi)}} \xrightarrow{d} \mathcal{N}(0,1)$$

The figure below shows this convergence for $\pi = 0.5$: at $n = 5$ the histogram is visibly discrete; by $n = 100$ it is indistinguishable from the Normal curve.



Bin(n, 0.5) (bars) vs. $\mathcal{N}(n/2,\ n/4)$ (curve) for $n = 5, 20, 100$.

The standardization step (center by μ, scale by σ) is what makes the convergence work: it aligns the Binomial with the $\mathcal{N}(0,1)$ template regardless of n and π.



Left: Bin(30, 0.5). Center: standardized $Z = (X - \mu)/\sigma$ with $\mathcal{N}(0,1)$ overlay. Right: limiting $\mathcal{N}(0,1)$.

Multivariate Gaussian $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$: for $\mathbf{X} \in \mathbb{R}^d$ with mean vector $\boldsymbol{\mu}$ and covariance matrix $\boldsymbol{\Sigma}$. Any linear transformation $\mathbf{AX} + \mathbf{b} \sim \mathcal{N}(\mathbf{A}\boldsymbol{\mu} + \mathbf{b},\, \mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^\top)$. The OLS estimator is a linear function of the Gaussian noise, which is why its sampling distribution is also Gaussian.

Statistics

Estimators: Bias, Variance, MSE

We observe data $X_1, \ldots, X_n$ drawn i.i.d. from a distribution with unknown parameter θ. An **estimator** $\hat{\theta} = g(X_1, \ldots, X_n)$ is a function of the data that approximates θ; its quality is measured by how close it tends to be.

The **bias** is the systematic error: $\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta$. An estimator is **unbiased** if $\mathbb{E}[\hat{\theta}] = \theta$. The **variance** $\text{Var}(\hat{\theta})$ measures how much $\hat{\theta}$ fluctuates across different samples. The **mean squared error** combines both:

$$\text{MSE}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \theta)^2] = \text{Bias}(\hat{\theta})^2 + \text{Var}(\hat{\theta})$$

This decomposition is fundamental to the course: a biased estimator with low variance (Ridge regression) can have lower MSE than an unbiased one with high variance (OLS in high dimensions).

Sample Statistics

Given observations $x_1, \ldots, x_n$:

Sample mean: $\bar{x} = \frac{1}{n}\sum_{i=1}^n x_i$, an unbiased estimator of $\mathbb{E}[X]$.

Sample variance: $s^2 = \frac{1}{n-1}\sum_{i=1}^n (x_i - \bar{x})^2$, an unbiased estimator of $\text{Var}(X)$. The $n-1$ denominator (Bessel's correction) compensates for using $\bar{x}$ instead of the true mean μ; estimating μ from the same data costs one degree of freedom.

Sample covariance: $\widehat{\text{Cov}}(X, Y) = \frac{1}{n-1}\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$.

Maximum Likelihood Estimation

Given i.i.d. data $x_1, \ldots, x_n$ from a distribution with density $p(x; \theta)$, the **likelihood** $L(\theta) = \prod_{i=1}^n p(x_i; \theta)$ measures how probable the observed data is under parameter θ. The **log-likelihood** converts the product to a sum:

$$\ell(\theta) = \sum_{i=1}^n \log p(x_i; \theta)$$

The **Maximum Likelihood Estimator (MLE)** is $\hat{\theta}_{\text{MLE}} = \arg\max_\theta \ell(\theta)$.

Example: Gaussian mean. Let $x_i \sim \mathcal{N}(\mu, \sigma^2)$ with σ^2 known. Then

$$\ell(\mu) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^n (x_i - \mu)^2 \tag{1}$$

Maximizing over μ is equivalent to minimizing $\sum_i (x_i - \mu)^2$. Setting $\partial\ell/\partial\mu = 0$:

$$\frac{1}{\sigma^2}\sum_{i=1}^n (x_i - \mu) = 0 \implies \hat{\mu}_{\text{MLE}} = \bar{x}$$

Minimizing squared error and maximizing the Gaussian log-likelihood are the same optimization problem. OLS is MLE under a Gaussian noise model.

Example: Bernoulli. Let $x_i \sim \text{Bernoulli}(\pi)$. The log-likelihood is $\ell(\pi) = \sum_i [x_i \log \pi + (1 - x_i)\log(1 - \pi)]$. Setting $d\ell/d\pi = 0$ gives $\hat{\pi}_{\text{MLE}} = \bar{x}$, the sample proportion of 1s. Logistic regression extends this by letting π depend on $\mathbf{x}$ through a sigmoid function.

Asymptotic properties. Under mild regularity conditions, as $n \to \infty$: the MLE is *consistent* ($\hat{\theta} \to \theta$ in probability), *asymptotically normal* ($\sqrt{n}(\hat{\theta} - \theta) \xrightarrow{d} \mathcal{N}(0, \mathcal{I}(\theta)^{-1})$), and *efficient* (achieves the Cramér-Rao lower bound), where $\mathcal{I}(\theta) = -\mathbb{E}[\partial^2 \log p(X; \theta)/\partial\theta^2]$ is the Fisher information.

The Learning Problem

Supervised Learning and Empirical Risk Minimization

In **supervised learning**, we are given a training set $D_n = \{(x_i, y_i)\}_{i=1}^n$ of input-label pairs drawn i.i.d. from an unknown joint distribution ν on $\mathcal{X} \times \mathcal{Y}$, and the goal is to find a function $f : \mathcal{X} \to \mathcal{Y}$ that predicts labels well on *new, unseen* inputs.

The nature of $\mathcal{Y}$ determines the problem type:

- **Regression**: $\mathcal{Y} = \mathbb{R}$, predict a continuous quantity
- **Classification**: $\mathcal{Y} = \{0, 1\}$ (binary) or $\{1, \ldots, K\}$ (multiclass), assign a discrete label

To measure how good a predictor f is, we fix a **loss function** $\ell(y, f(x)) \geq 0$ and define the **true risk** as the expected loss on a fresh draw: $\mathcal{R}(f) = \mathbb{E}_{(x,y)\sim\nu}[\ell(y, f(x))]$. Since ν is unknown, we minimize the **empirical risk** instead:

$$\hat{\mathcal{R}}_n(f) = \frac{1}{n} \sum_{i=1}^{n} \ell(y_i, f(x_i))$$

Definition 2 (Empirical Risk Minimizer)**.** *The **ERM** over a hypothesis class $\mathcal{F}$ is $\hat{f} = \arg\min_{f \in \mathcal{F}} \hat{\mathcal{R}}_n(f)$.*

The fundamental question: when does minimizing $\hat{\mathcal{R}}_n$ also minimize $\mathcal{R}$? The gap $\mathcal{R}(\hat{f}) - \hat{\mathcal{R}}_n(\hat{f})$ is the generalization gap, and controlling it is the central theme of the bias-variance tradeoff.

Generalization and the Bayes Optimal Predictor

Generalization is how well a model trained on D_n performs on new data. A model that fits the training data perfectly but fails on unseen inputs has *overfit*; a model that cannot even fit training data has *underfit*.

The best possible predictor, ignoring computational constraints and the fact that ν is unknown, is the **Bayes optimal predictor**: $f^* = \arg\min_f \mathcal{R}(f)$. Under squared loss, $f^*(x) = \mathbb{E}[y \mid x]$, as derived in the linear regression notes. Its risk is the **Bayes risk** $\mathcal{R}^* = \mathbb{E}_x[\mathrm{Var}(y \mid x)]$, the irreducible noise floor that no predictor can beat. The prediction error of any estimator $\hat{f}$ decomposes as:

$$\boxed{\mathcal{R}(\hat{f}) = \underbrace{\mathcal{R}^*}_{\text{irreducible}} + \underbrace{\mathrm{Bias}^2 + \mathrm{Var}}_{\text{reducible}}}$$

Loss Functions

The choice of loss function determines what "good prediction" means, and different losses lead to fundamentally different optimization problems.

Squared Loss

$$\ell(y, \hat{y}) = (y - \hat{y})^2$$

The standard loss for regression. Convex and differentiable everywhere, with the Bayes optimal predictor $f^*(x) = \mathbb{E}[y \mid x]$. As shown above, minimizing squared loss is equivalent to Gaussian MLE. The downside is sensitivity to outliers, since large errors are penalized quadratically.

0-1 Loss and Convex Surrogates

$$\ell_{\text{0-1}}(y, \hat{y}) = \mathbf{1}[\hat{y} \neq y]$$

Counts mistakes. The Bayes classifier under 0-1 loss is $f^*(x) = \arg\max_k P(y = k \mid x)$, but 0-1 loss is non-convex and non-differentiable, making direct minimization NP-hard even for linear classifiers. Logistic regression and cross-entropy replace it with convex, differentiable surrogates that are tractable to optimize.

Logistic Loss and Cross-Entropy

For binary classification with $y \in \{0, 1\}$ and a model outputting $\hat{p} = \sigma(\mathbf{x}^\top \boldsymbol{\theta}) \in (0, 1)$ where $\sigma(t) = 1/(1 + e^{-t})$ is the sigmoid:

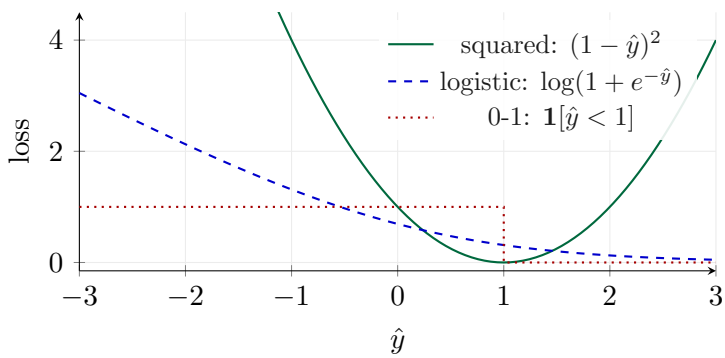
$$\ell_{\text{logistic}}(y, \hat{p}) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p})$$

Maximizing the Bernoulli likelihood equals minimizing this loss. It is convex in $\boldsymbol{\theta}$, differentiable everywhere, and penalizes confident wrong predictions heavily.

The multiclass generalization is **cross-entropy**: $\ell_{\text{CE}}(y, \hat{\mathbf{p}}) = -\log \hat{p}_y$, where $\hat{\mathbf{p}} = \text{softmax}(\mathbf{z})$ maps scores $\mathbf{z} \in \mathbb{R}^K$ to probabilities via $\text{softmax}(\mathbf{z})_k = e^{z_k} / \sum_j e^{z_j}$.

Why Convexity of Losses Matters

A loss ℓ that is convex in the model parameters $\boldsymbol{\theta}$ guarantees that any local minimum of $\hat{\mathcal{R}}_n$ is a global minimum, so gradient descent is guaranteed to converge to the best solution in the hypothesis class. Non-convex losses (0-1 loss, or the loss landscape of a neural network) can have many local minima, and gradient-based methods may converge to suboptimal solutions.



Generative vs. Discriminative Models

There are two philosophically different approaches to supervised learning. The joint distribution $p(x, y)$ factorizes in two ways:

$$p(x, y) = \underbrace{p(y \mid x)}_{\text{discriminative}} \cdot p(x) = \underbrace{p(x \mid y)}_{\text{class-conditional}} \cdot p(y)$$

Definition 3 (Discriminative Model)**.** *A* ***discriminative model*** *directly models $p(y \mid x)$: given input x, what is the label? Training maximizes the conditional likelihood $\sum_i \log p(y_i \mid x_i)$.*

Definition 4 (Generative Model)**.** *A* ***generative model*** *models $p(x \mid y)$ and $p(y)$, then uses Bayes' rule $p(y \mid x) = p(x \mid y)\, p(y)/p(x)$ for prediction. Training maximizes the joint likelihood $\sum_i \log p(x_i, y_i)$.*

Discriminative models (linear regression, logistic regression, neural networks) learn the decision boundary directly and tend to perform better with large n, since they only model what is needed for prediction. Generative models (Gaussian Discriminant Analysis, Naive Bayes) model the full data-generating process, which requires stronger assumptions but can be more data-efficient when those assumptions hold.

Approach	**Models**	**Training objective**
Logistic regression (disc.)	$p(y \mid x)$ directly	$\max \sum_i \log p(y_i \mid x_i; \boldsymbol{\theta})$
Gaussian Discrim. Analysis (gen.)	$p(x \mid y)$ and $p(y)$	$\max \sum_i \log p(x_i, y_i; \boldsymbol{\theta})$